

Université Paris Cité

Faculté des Sciences

UFR Mathématiques et Informatique

Master 1 Informatique

Représentation des connaissances et raisonnement

Argumentation computationnelle

Rapport de Projet

Développement d'un Système d'Argumentation

Fait par :

Merabti Amar (DCI)

Souayah Abdelkader (VMI)

Année universitaire : 2024/25

Table des matières

1. Introduction	2
2. Contexte et objectifs.....	3
3. Architecture du programme.....	3
3.1. Lecture du Framework d'argumentation	3
3.2. Vérifications et Opérations sur les Ensembles	3
3.3. Extensions et Acceptabilité	3
4. Description des algorithmes.....	4
4.1. Lecture et construction du Framework d'argumentation	4
4.2. Identification des ensembles sans conflit	4
4.3. Calcul des extensions complètes et stables	5
Extensions complètes.....	5
Extensions stables	5
4.4. Décision d'acceptabilité.....	5
Crédule	5
Sceptique.....	5
5. Structures de données utilisées	5
6. Complexité et approche utilisée	6
7. Interface utilisateur	6
8. Conclusion	8
Références bibliographiques	9

1. Introduction

L'argumentation computationnelle représente un domaine au sein de l'intelligence artificielle qui s'intéresse à la modélisation et à l'analyse des interactions argumentatives. Cette discipline a des applications variées, telles que la prise de décision, les systèmes multi-agents et la résolution de conflits. Ce projet s'inscrit donc dans ce contexte et a pour objectif de développer un outil capable de traiter des cadres d'argumentation abstraits en s'appuyant sur différents critères et méthodes. En intégrant des concepts essentiels, comme les extensions complètes et stables, cette initiative propose une assise pratique pour bien comprendre les mécanismes du raisonnement et aussi la représentation des connaissances.

2. Contexte et objectifs

L'objectif de ce projet est de développer un outil permettant de résoudre des problèmes liés à l'argumentation computationnelle. Nous avons implémenté un système capable de traiter un Argumentation Framework (AF), représenté par, où :

- Un ensemble d'arguments abstraits.
- Un ensemble des attaques entre ces arguments (relation d'attaque).

Les problèmes traités sont les suivants :

- a. **SE-CO** : Trouver une extension complète.
- b. **DC-CO** : Déterminer si un argument appartient à une extension complète.
- c. **DS-CO** : Déterminer si un argument appartient à toutes les extensions complètes.
- d. **SE-ST** : Trouver une extension stable.
- e. **DC-ST** : Déterminer si un argument appartient à une extension stable.
- f. **DS-ST** : Déterminer si un argument appartient à toutes les extensions stables.

3. Architecture du Programme

Notre programme est implémenté en Python et comporte les modules principaux suivants :

1. Lecture du Framework d'Argumentation :

- Lecture des arguments et des attaques à partir d'un fichier texte sous le format "apx"
- Validation pour détecter des erreurs telles que les auto-attaques.

2. Vérifications et Opérations sur les Ensembles :

- Génération de sous-ensembles d'arguments (power set).
- Tests pour vérifier si un sous-ensemble est sans conflit ou s'il défend tous ses arguments.

3. Extensions et Acceptabilité :

- Calcul des extensions complètes et stables.
- Vérification de l'acceptabilité crétule et sceptique.

4. Description des algorithmes

1. Lecture et construction du Framework d'argumentation

L'algorithme lit un fichier contenant des définitions d'arguments (ligne "arg(arg_name).") et des attaques (ligne "att(arg1, arg2).") pour construire les ensembles d'arguments et l'ensemble des relations d'attaque.

Structures de données :

- Arguments : Ensemble (évitant les doublons).
- Attaques : Liste de tuples pour capturer les relations d'attaque.

Traitement des erreurs :

- Arguments non définis.
- Auto-attaques (non autorisées).
- Fichier manquant ou mal formé.

2. Identification des ensembles sans conflit

Cet algorithme vérifie que pour tout sous-ensemble S , aucun argument de S n'attaque un autre argument de S .

Approche :

- Génération de toutes les combinaisons possibles (à l'aide de « powerset »).
- Validation des relations d'attaque pour chaque sous-ensemble.

Optimisation : L'utilisation de générateurs permet de réduire l'empreinte mémoire, ce qui est essentiel pour traiter de grands ensembles d'arguments.

3. Calcul des extensions complètes et stables

a. Extensions complètes

L'algorithme examine si chaque argument appartenant à un sous-ensemble est effectivement soutenu par d'autres arguments dans ce sous-ensemble. Cette analyse garantit que tous les arguments dans l'ensemble sont en conformité avec les règles d'admissibilité définies pour le cadre d'argumentation.

Tous les arguments qu'un sous-ensemble peut défendre sont systématiquement intégrés afin d'assurer une conformité totale avec les critères d'admissibilité.

b. Extensions stables

En plus de l'admissibilité, l'algorithme s'assure que tous les arguments extérieurs à l'ensemble donné sont attaqués.

4. Décision d'acceptabilité

- a. **Crédule** : Un argument est crédulement accepté s'il figure dans au moins une extension.
- b. **Sceptique** : Un argument est sceptiquement accepté s'il figure dans toutes les extensions possibles.

5. Structure de données utilisées

- **Ensembles** : Utilisés pour stocker les arguments sans doublons.
- **Listes** : Représentent les attaques et extensions.
- **Générateurs** : Optimisation pour générer des sous-ensembles.

Les choix de structures de données permettent une manipulation efficace des relations d'attaque et des calculs sur les extensions.

6. Complexité et Approche Utilisée

La complexité des algorithmes implémentés est dominée par la génération des sous-ensembles, qui a une complexité exponentielle 2^n , où n est la taille de l'ensemble des arguments. Cela est inévitable pour garantir l'exhaustivité des résultats dans le cadre des approches théoriques que nous avons adoptées.

Afin de simplifier la mise en œuvre et de garantir la conformité aux définitions théoriques, nous avons opté pour des approches naïves et directes. Ces approches consistent à examiner explicitement tous les sous-ensembles et à appliquer des tests systématiques pour identifier les extensions ou vérifier les critères d'acceptabilité. Bien que ces méthodes soient gourmandes en calcul, elles permettent d'éviter toute approximation ou perte de précision dans les résultats.

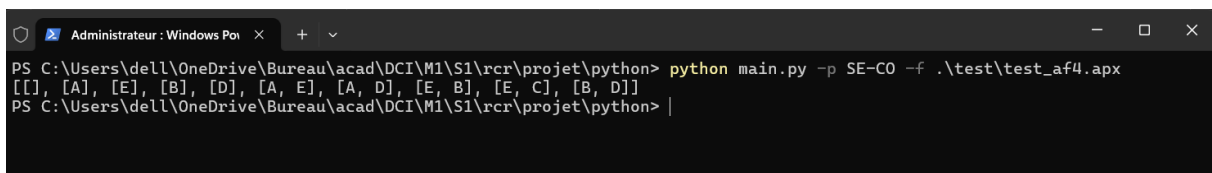
7. Interface utilisateur

L'outil propose une interface en ligne de commande avec les options suivantes :

- **-p** : Spécifie le type d'analyse : « **SE-CO** », « **SE-ST** », « **DC-CO** », « **DC-ST** », « **DS-CO** », « **DS-ST** ».
- **-f** : Chemin du fichier contenant l'AF.
- **-a** : Argument à analyser pour les requêtes « **DC-XX** » ou « **DS-XX** ».

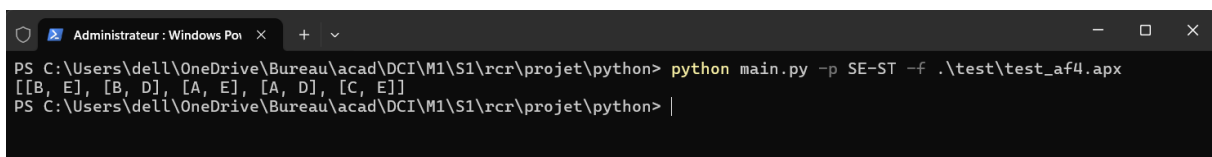
Exemple d'exécution sur le fichier de test : 'test_af4.apx' et avec l'argument 'A'

a. **SE-CO** :



```
Administrateur : Windows Po... x + v
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> python main.py -p SE-CO -f .\test\test_af4.apx
[[], [A], [E], [B], [D], [A, E], [A, D], [E, B], [E, C], [B, D]]
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> |
```

b. **SE-ST** :



```
Administrateur : Windows Po... x + v
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> python main.py -p SE-ST -f .\test\test_af4.apx
[[B, E], [B, D], [A, E], [A, D], [C, E]]
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> |
```

c. **DC-CO** :



```
Administrateur : Windows Po... x + v
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> python main.py -p DC-CO -f .\test\test_af4.apx -a A
YES
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python>
```

d. **DC-ST :**

```
Administrateur : Windows Po... x + v - □ x
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> python main.py -p DC-ST -f .\test\test_af4.apx -a A
YES
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> |
```

e. **DS-CO :**

```
Administrateur : Windows Po... x + v - □ x
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> python main.py -p DS-CO -f .\test\test_af4.apx -a A
NO
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> |
```

f. **DS-ST :**

```
Administrateur : Windows Po... x + v - □ x
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> python main.py -p DS-ST -f .\test\test_af4.apx -a A
NO
PS C:\Users\dell\OneDrive\Bureau\acad\DCI\M1\S1\rcr\projet\python> |
```

On a testé sur tout l'ensemble des fichiers de teste et on a eu des résultats corrects.

8. Conclusion

Ce projet a permis de mettre en oeuvre un système complet d'analyse de frameworks d'argumentation, en répondant de manière exhaustive aux problématiques posées par les différentes sémantiques étudiées. Les algorithmes implémentés respectent les définitions théoriques et fournissent des résultats précis. Nous avons également intégré une interface utilisateur claire et robuste, conçue pour faciliter l'exécution des différentes analyses tout en optimisant l'expérience utilisateur. Cette interface permet une interaction rapide, rendant accessible l'analyse des frameworks d'argumentation abstraits à travers des commandes simples et intuitives.

Références bibliographiques :

1. Baroni, P., Caminada, M., & Giacomin, M. (2011). An introduction to argumentation frameworks. *The Knowledge Engineering Review*.
2. Dung, P. M. (1995). On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*.